

8 旅行商与车辆路径入门：约束多到写不完时怎么办

一辆车走遍若干个货位再回到仓库，是最经典的路线问题。它的模型只比第 5 章的指派多一个要求：所有的弧要连成一个圈。这一个要求需要指数多的约束。这一章看两种应对：不一次写完、需要时再加（割平面），和引入辅助变量把它压缩成多项式个约束（MTZ），并把载重加进来，得到车辆路径问题的骨架。

学完本章，你能

- 1. 写出旅行商的整数模型，说明只保留出入度约束就是第 5 章的指派问题，它的解为什么是一堆互不相连的小回路。
- 2. 写出子回路消除约束，用最大流分离被违反的约束，手写根节点的割平面循环。
- 3. 写出 MTZ 模型，说明它为什么规模小但 LP 下界弱。
- 4. 写出带载重的车辆路径（CVRP）的模型骨架，看出它由指派、装载和旅行商拼成；会用最近邻和 2-opt，并知道局部搜索停在哪里。

本章信息

用时	阅读与交互 80-110 分钟，Lab 3 小时左右
前置	第 4 章（割平面、惰性约束）、第 5、6、7 章（指派、最大流与最小割、覆盖割与载重）。会读 Python 代码。
对拍	SciPy 1.17.1 · HiGHS 1.12.0；本章的算法由页面内置代码实现，不依赖求解器。
配套	Lab 代码包：5 个练习 + 1 个挑战，含手写的 Held-Karp、割平面循环、车辆路径的子集动态规划，pytest 自动判分

PART 1 数学规划基础	PART 2 组合优化	PART 3 调度理论	PART 4 路径与波次
PART 5 大规模方法	PART 6 启发式	PART 7 多智能体时空规划	PART 8 动态与不确定

第二部分 组合优化：指派问题与匈牙利算法 → 最短路与网络流 → 带容量的指派与背包 → 旅行商与车辆路径入门（本章）。

8.1 旅行商与指派松弛

一辆车从仓库出发，要走遍 8 个货位，再回到仓库，总行驶距离最小。这就是旅行商问题 (TSP)。它是仓库拣选路径、巡检路线、补货路线的共同骨架。

设 $x_{ij} = 1$ 表示车从点 i 直接开到点 j 。点 0 是仓库，一共 $n = 9$ 个点：

TSP $\min \sum c_{ij}x_{ij}, \quad \sum_j x_{ij} = 1, \quad \sum_i x_{ij} = 1, \quad x_{ij} \in \{0, 1\}$ ，还要连成一个圈

前三项是出入度约束；最后一项要另外写

前面的部分你已经见过：每个点选一个后继，每个点只被选一次，这就是第 5 章的**指派问题**（只不过“点选后继”，且不许选自己）。它有全单模的约束矩阵，LP 直接给整数解，用匈牙利算法就能解。

缺的是“连成一个圈”。指派问题只保证每个点有一进一出，不保证所有点在同一个圈里，它的解是一堆互不相连的小回路（置换的圈分解）。把“连成一个圈”这一条去掉，就是 TSP 的一个松弛，最优值是 TSP 的下界。

先预测 9 个点（仓库 + 8 个货位）。只保留“每个点一条出弧、一条入弧”的约束（第 5 章的指派问题），它的最优解会把这 9 个点分成几个互不相连的回路？

1 个：正好是一条完整的回路

2 个

4 个

8 个

预测正确。 4 个： $\{\text{仓}, 2\}$ 、 $\{1, 7, 8\}$ 、 $\{3, 5\}$ 、 $\{4, 6\}$ ，总长 198。指派问题只管每个点有一进一出，不管连不连成一个圈，所以偏向短小的圈。最优回路是 255。下面的图已经解锁。

显示

指派松弛的解

最优回路

指派松弛

198

只有出入度约束（下界）

最优回路

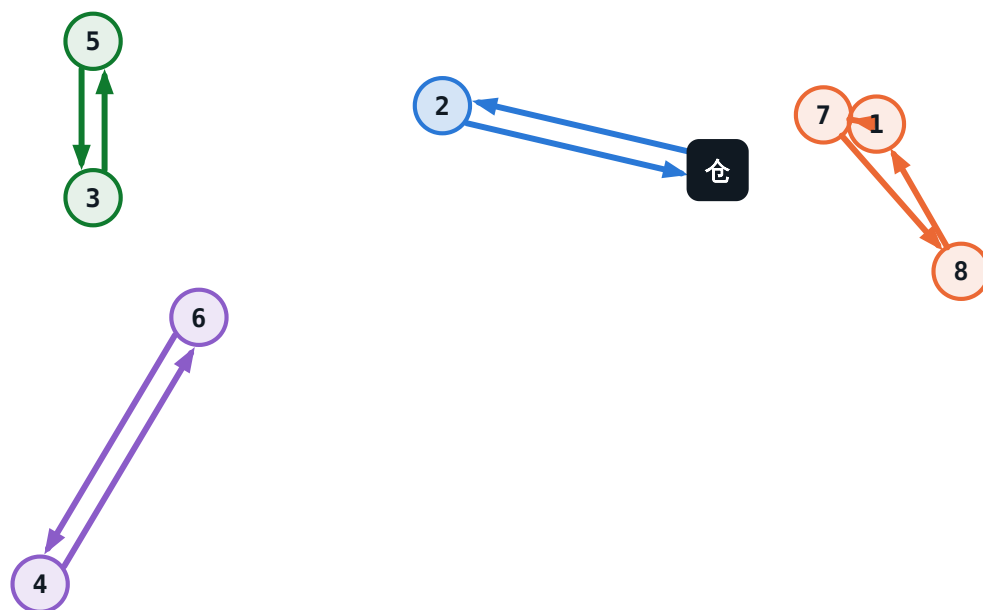
255

Held-Karp 精确解

回路个数

4

应当是 1



指派松弛的最优解，总长 **198**，把 9 个点分成了 **4** 个互不相连的回路： $\{\text{仓库}, 2\}$ 、 $\{1, 7, 8\}$ 、 $\{3, 5\}$ 、 $\{4, 6\}$ 。每个点都有一条出弧和一条入弧，约束都满足，但它不是一条走遍所有点的回路。

这就是第 5 章的指派问题：“点 i 的后继是谁”，每个点选一个后继，后继不重复，不许选自己。它没有规定“连成一个圈”，所以最优解自然偏向短小的圈。

图 8-1 仓库（方块）与 8 个货位，距离是取整的直线距离。“指派松弛的解”只有出入度约束（第 5 章的指派问题，页面内置单纯形法求解），不同颜色是不同的回路；“最优回路”由 Held-Karp 动态规划得到。

读图

指派松弛：198，四个回路。 $\{\text{仓库}, 2\}$ 、 $\{1, 7, 8\}$ 、 $\{3, 5\}$ 、 $\{4, 6\}$ ：相邻的几个点两两互为后继，形成一个个小圈。每个点的出入度都是 1，约束全满足，但没有一辆车能这样开：它们互不相连。

最优回路：255。差的 57 就是“所有点必须连成一个圈”的代价。用下界的语言说：这个松弛给出的下界 198，比最优低了 22%，太松，只靠它不能剪枝，需要把“连成一个圈”写进模型。

精确解可以用动态规划（Held-Karp）： $dp[S][k]$ 是从仓库出发走遍集合 S 、停在 k 的最短路程，时间 $O(n^2 2^n)$ 。9 个点几乎是瞬间，20 个点已经需要几十亿次运算，再多就不行了。这也是为什么需要整数规划的方法，而不是枚举。

8.2 子回路消除的割平面

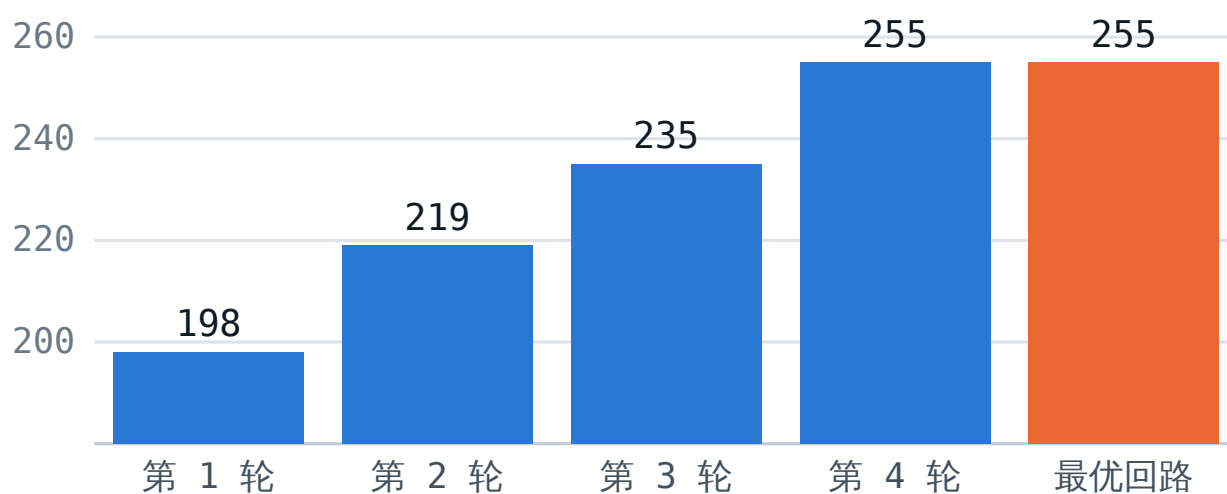
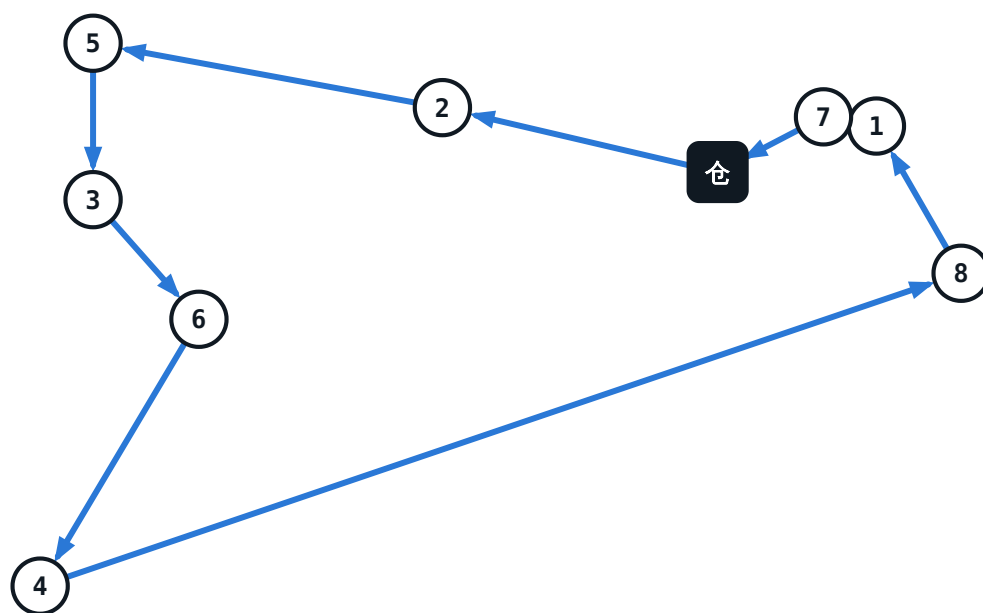
要把“连成一个圈”写成线性约束，办法是禁止每一个小圈：对不含仓库的任意点集 S ，圈内的弧不能把 S 里的点连成闭环。

子回路消除 $\sum_{i \in S, j \in S, i \neq j} x_{ij} \leq |S| - 1$ ，对所有 $S \subseteq \{1, \dots, n-1\}, |S| \geq 2$ S 里 $|S|$ 个点，最多有 $|S|-1$ 条弧留在 S 里；否则就闭合成圈

这样的 S 有 $2^{n-1} - n$ 个，9 个点就有 247 个，30 个点超过五亿。一次写完不现实。办法和第 7 章的覆盖割一样：先不写，需要时再加。

- 1 只用出入度约束解 LP。
- 2 分离：找 LP 解违反了哪些子回路约束。
- 3 把找到的约束加进去，再解，回到第 2 步；找不到违反的约束就停。

分离怎么做？出入度约束成立时，子回路约束等价于“有弧从 S 外面进来”： S 内部的弧数 = $|S|$ - 进入 S 的弧数，所以内部的弧至多 $|S| - 1$ ，当且仅当进入 S 的弧数（的和）至少 1。换句话说：从仓库到 S 里的每个点，都要能送过去 1 个单位的流量。这正是第 6 章的最大流：把 LP 解的 x_{ij} 当作弧的容量，对每个点 t 求仓库到 t 的最大流；流量小于 1，最小割里汇点那一侧的点集就是被违反的 S 。这对分数解也成立，一次最大流就能找到最违反的那一组。



第 4 轮的 LP : **255**，带着已加入的 6 条子回路约束。

仓库到每个点的最大流都是 1，再没有被违反的子回路约束。LP 的解是一条完整的回路，值 255 = 最优值 255。总共只加了 6 条约束，完整的子回路约束有 247 条。

图 8-2 根节点的割平面循环。左：这一轮 LP 的解（虚线是分数弧）；橙色圈住的点集是这一轮分离出来的子回路约束。右：下界的变化，最右边是最优回路 255。

读图

第 1 轮就是 8.1 节的指派松弛，198，四个回路。仓库所在的回路 {仓, 2} 里的点当然从仓库可达，但 {3, 5}、{4, 6}、{1, 7, 8} 三组点被孤立在外面：仓库到它们的最大流是 0，各产生一条约束。

第 2 轮LP 上升到 219，重新连出了几条更长的弧，出现了新的小圈 {1, 7} 和一大块 {2, 3, 4, 5, 6}；**第 3 轮**235，再分离出 {3, 4, 5, 6}；**第 4 轮**255，仓库到每个点的最大流都是 1，LP 解是一条完整的回路，等于最优。一共只加了 6 条约束，完整的有 247 条。

这个例子里每一轮 LP 的解碰巧都是整数点。一般的实例上，子回路 LP 的最优解还可能带分数，这时下界（Held-Karp 下界）通常离最优只差百分之一到二，剩下的差距交给分支。求解器里这就是**惰性约束**（lazy constraints）：约束不预先写入，解出整数解后再检查、再加。

8.3 MTZ 与模型的强弱

另一条路是把指数多的约束压成多项式个，代价是加辅助变量。最常用的是 Miller-Tucker-Zemlin (MTZ, 1960)：给每个客户一个数 u_i ，表示它是路线上的第几站。

$$\text{MTZ} \quad u_i - u_j + m x_{ij} \leq m - 1, \quad 1 \leq u_i \leq m \quad m \text{ 是客户数}; i, j \text{ 是客户}, i \neq j$$

读法：如果 $x_{ij} = 1$ （车从 i 直接开到 j ），约束变成 $u_j \geq u_i + 1$ ：沿着路线，站序号严格递增。如果一个不含仓库的小圈存在，沿圈走一周，站序号要一路递增又回到起点，不可能，所以小圈被排除。如果 $x_{ij} = 0$ ，约束变成 $u_i - u_j \leq m - 1$ ，永远成立，不起作用。

约束只有约 m^2 条，可以直接交给求解器。但这是第 3 章的 Big-M：系数 m 是“为了让约束在 $x = 0$ 时失效”取的大数，LP 松弛里 x_{ij} 可以取很小的分数，约束几乎不起作用，下界很弱。

先预测 8 个客户的例子：指派松弛的下界是 **198**，最优是 **255**。**MTZ 紧凑模型的 LP 下界大约是多少？**

约 203，只比指派松弛好一点

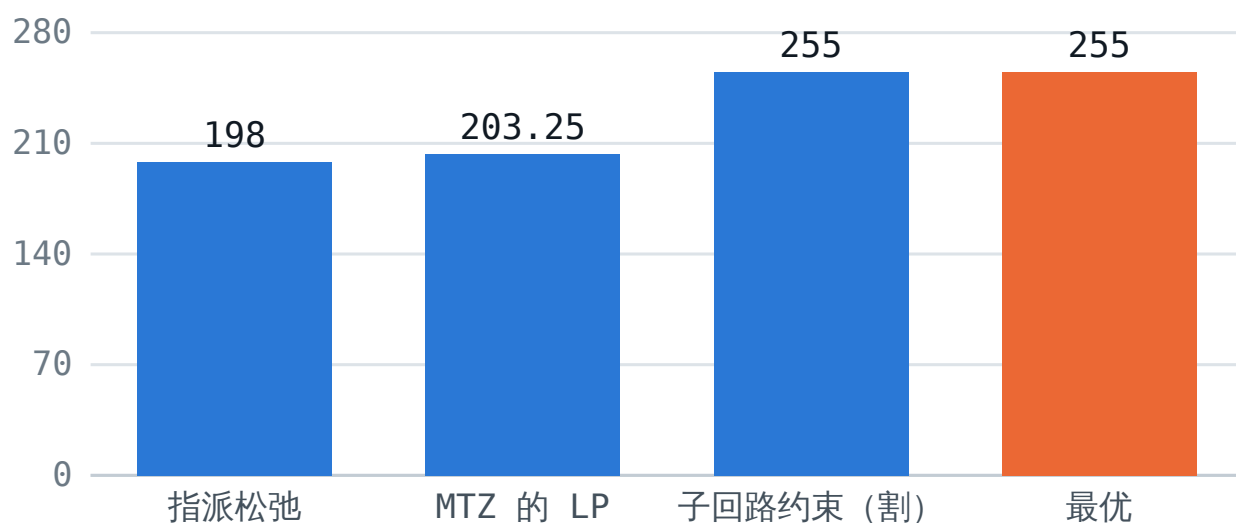
约 225

约 250

255，与最优相等

预测正确。 203.25。MTZ 的约束里有 Big-M 系数 m ，LP 里 x 取很小的分数时约束几乎不起作用，下界很弱。用割平面加子回路约束，下界可以到 255。下面的图已经解锁，拖动客户数看别的规模。

客户数 $m = 8$



模型	变量	约束	下界	差
指派松弛	72	18	198	57
MTZ	81	83	203.25	51.75
子回路 (惰性)	72	18 + 6	255	0

9 个点 (仓库 + 8 个客户)。最优 255。指派松弛 198，MTZ 的 LP 203.25，子回路约束的 LP 255 (割平面循环 4 轮，共 6 条)。

下界的顺序总是：指派松弛 $\leq$ MTZ $\leq$ 子回路约束 $\leq$ 最优。MTZ 的约束数是 n 的平方级，但 LP 很弱；完整的子回路约束有 247 条，实际只加了 6 条 (表里“18 + 6”是出入度约束加惰性加入的约束)。

图 8-3 三种模型的下界 (同一组点，取前 m 个客户)。指派松弛只有出入度约束；MTZ 是紧凑模型的 LP 松弛；“子回路约束”是割平面循环停下时的 LP 值。最右边是最优。全部由页面内置的单纯形法、割平面循环和动态规划算出。

读图

8 个客户：指派松弛 198，MTZ 的 LP 203.25，子回路约束的 LP 255，最优 255。MTZ 比指派松弛只高了 5，几乎没有帮助；割平面循环把下界一直推到最优。把客户数拖到 4：168、170、211、211，同样的格局。

两种模型的取舍。MTZ 好写、规模小，适合客户少、要快速建模的场合；割平面模型的 LP 强得多，是求解器解大规模旅行商的做法，但要有惰性约束和分离程序。带时间窗、多车、载重的变种

里，MTZ 的辅助变量还可以直接表达“到达时间”，这是它常被保留的原因（第 15 章）。可以对 MTZ 做提升（Desrochers-Laporte）收紧下界，仍然不及子回路约束。

选模型看的是 LP 下界，不是约束条数

约束少不等于好解。第 3 章讲的紧化在这里又出现了一次：同一个整数集合，可以有强弱不同的 LP 松弛。指派 $\leq$ MTZ $\leq$ 子回路约束 $\leq$ 最优，这个顺序对任何实例都成立，Lab 里在随机实例上验证。

8.4 车辆路径的骨架

现实里很少只有一辆车。把 8 个货位的托盘（每个货位的托盘数不同）分给几辆载重为 9 的车，每辆车从仓库出发、送完再回来，总行驶距离最小。这是带载重的车辆路径问题（CVRP）。

它是这门课前面几章的拼装：

指派（第 5 章）

每个客户恰有一辆车到达、一辆车离开：出入度约束，和 TSP 一样。仓库的出度是车数 K 。

装载与载重（第 7 章）

每条路线的托盘数不超过 Q 。写成割：对客户集合 S ，进入 S 的弧数至少是 $\lceil d(S) / Q \rceil$ ， $d(S)$ 是 S 的托盘总数。这个上取整就是装箱的下界。

子回路（本章）

$\lceil d(S) / Q \rceil \geq 1$ ，所以这类容量割把 8.2 节的子回路约束包含进来：不经过仓库的小圈，同样被排除。

求解的思路也是两种，对应两种分解：**先分组、再排路线**（cluster-first, route-second）：先把客户分给各辆车（广义指派，第 7 章），再对每组求旅行商；或者**先排一条大回路、再切开**（route-first, cluster-second）。扫描法（Gillett & Miller, 1974）是最简单的先分组：客户按相对仓库的角度排序，依次装车，装不下就换新车。

先预测 22 个托盘、每辆车载重 9、最多 3 辆车。最优的总行驶距离是 360。扫描法（按角度分组、依次装车，每组精确排路线）的结果，比最优多百分之几？

不到 1%

约 5%

约 16%

约 50%

预测正确。 419，比最优 360 多 59，约 16%。扫描法只看角度，不看距离，也不平衡各车的负荷：前两辆装满 9 个，第三辆只有 4 个。下面的图已经解锁。

方案

最优

扫描法

总行驶距离

360

最优（子集动态规划）

路线数

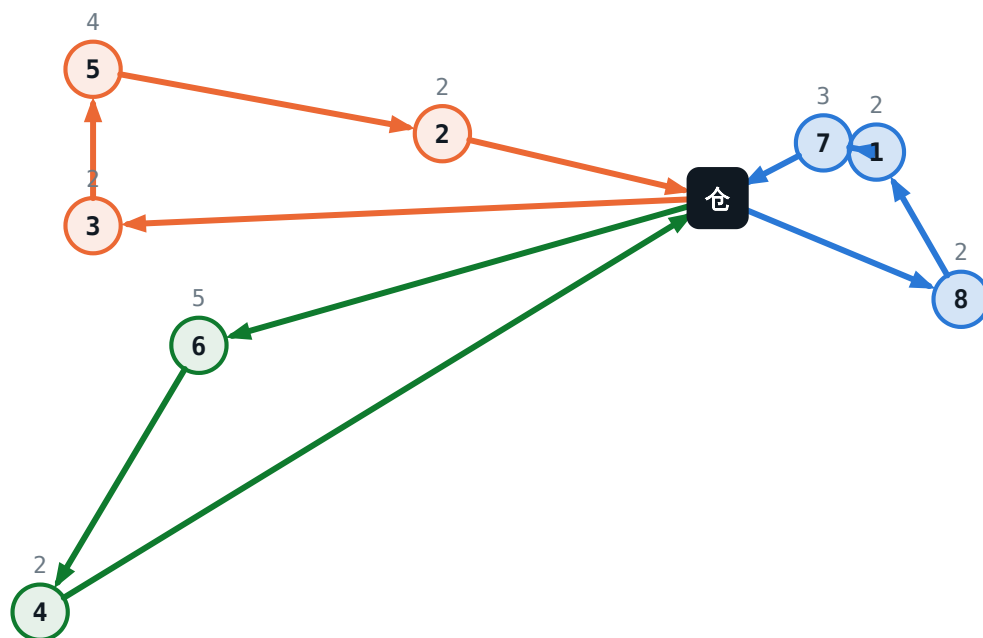
3

至少 $\lceil 22 \div 9 \rceil = 3$

对比：不限载重的单车回路

255

同一批点，一辆车走完



- 车 1：仓 → 8 → 1 → 7 → 仓 距离 60，托盘 7 / 9
- 车 2：仓 → 3 → 5 → 2 → 仓 距离 137，托盘 8 / 9
- 车 3：仓 → 6 → 4 → 仓 距离 163，托盘 7 / 9

最优的分组把相近的客户放在一起，三条路线的托盘数是 7、8、7，都不超载重。

图 8-4 带载重的多车路线：客户旁边的小数字是托盘数，每辆车载重 9，最多 3 辆。“最优”由子集动态规划得到（先算每个客户子集的最短单车路线，再划分）；“扫描法”按角度分组，每组用精确算法排路线。

读图

最优：360，三辆车。 22 个托盘，每车至多 9，至少要 $\lceil 22 / 9 \rceil = 3$ 辆。三条路线的距离是 60、137、163，托盘数 7、8、7。把同一批点交给一辆不限载重的车，只要 255；载重把它撕成三段，多花了 105。

扫描法：419，比最优多 59 (+16%)。它把客户按角度分成了托盘数 9、9、4 的三组，前两辆车装满，第三辆车几乎空着，最后一条路线单独跑了 122。最优的分组三车的负荷更均衡，也更按空间聚成团。这也说明，分组的质量决定了最终的路线，正是第 7 章广义指派要认真求解的原因。

子集动态规划要枚举 2^m 个客户子集，8 个客户瞬间完成，25 个以上就不现实。现实规模的 CVRP 用的是割平面加分支（分支切割，容量割 + 子回路割）、列生成（第 17、18 章）和启发式（第 21、22 章）。

8.5 启发式与局部搜索

精确方法在规模上有限。实际系统里最常用的是“先构造一个还行的解，再局部改进”。

最近邻：从仓库出发，每次去离当前点最近的没去过的点。它快、好写，但没有质量保证，最后常常被迫走很远的回头路。**2-opt**：选回路里的一段，整段反过来走，去掉两条弧、换上两条新弧；只要这样换能缩短路线，就换，直到没有任何一次反转能缩短路线为止。这样的解叫**局部最优**（对“反转一段”这一类邻域）。

当前回路

290

最近邻

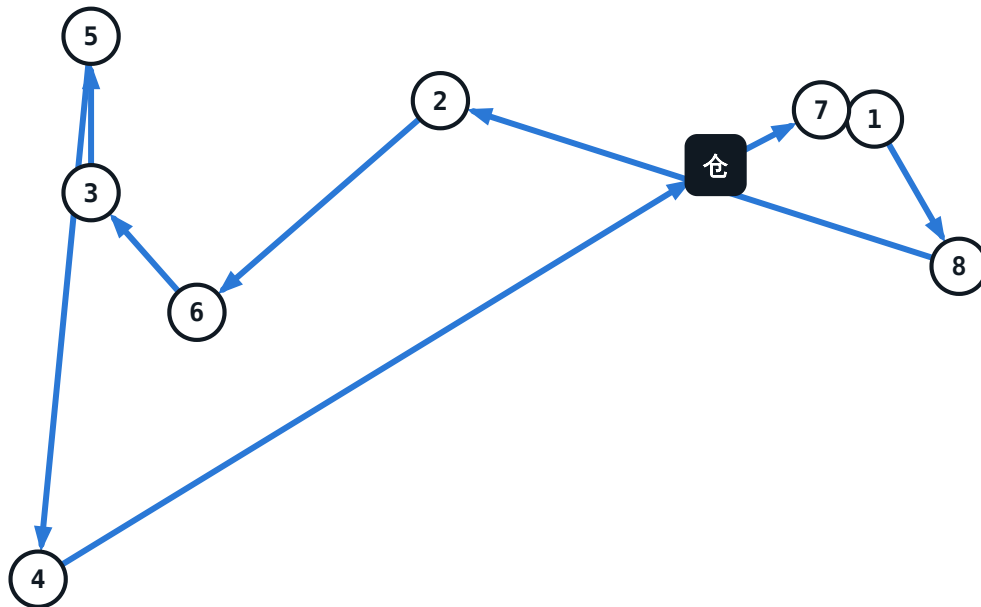
最优

255

Held-Karp

比最优长

13.7%



最近邻：从仓库出发，每次去离当前点最近的没去过的点。走到最后被迫回到很远的地方。

图 8-5 最近邻 → 2-opt → 最优。橙色是这一步新换上的弧。2-opt 按固定的顺序扫描（先找到的第一个改进就用），到没有改进为止。最后一张是精确解，用来局部最优离全局最优还有多远。

读图

最近邻 290，比最优 255 长 13.7%。**2-opt 两步**：第一步只省了 3 (287)，第二步省了 25 (262)，之后没有任何一次反转能缩短路线。**262**，比最优还长 **2.7%**。局部搜索停在一个“山谷”里，从这里出发，任何小改动都会变差，要得到 255 需要一次跨度更大的重排。

这是第六部分的主题：更大的邻域 (Or-opt、Lin-Kernighan)、允许暂时变差 (模拟退火、禁忌搜索)、破坏与修复 (ALNS)，都是在跳出这样的山谷。同时用下界 (本章的子回路 LP) 告诉你离最优最多还差多少：这里下界等于 255，262 至多比最优差 2.7%。对于度量 (满足三角不等式) 的旅行商，有近似算法保证：Christofides (1976) 的回路不超过最优的 1.5 倍。

8.6 在项目里的位置

路线问题在你的场景里有几个化身：仓库里的拣选路径、多个出入库口的补货巡回、发运波次的车辆装车与送货。它们的模型都长成本章的样子，差别在图和约束上。

仓库拣选路径

货位在巷道里，距离不是直线，而是网格图上的最短路（第 6 章）。先算好点对之间的最短路距离，再对要访问的点求旅行商。巷道结构好的时候有专用的规则（S 形、回转、最大间隙），第 14 章会讲。

多车、载重、时间窗

CVRP 加上每个客户的时间窗就是 VRPTW，是第 15 章的主题。MTZ 的辅助变量在这里很有用：把“第几站”换成“到达时间”，时间窗就是变量的上下界。

求解方式

几十个点、要证书：MIP 求解器加惰性约束。几百上千个点、要快：先用构造启发式和局部搜索（第 21 章），再用求解器的固定并优化（第 23 章）。规模再大，用列生成的“路线”模型（第 17 章）。

本章想留下的方法论：**约束的个数不是问题，分离才是**。只要能高效地找到当前解违反的那一条约束（这里是一次最大流），指数多的约束就可以“按需加”。这个想法在 Benders 分解（第 19 章）、分支定价（第 18 章）里会反复出现。

8.7 小结与 Lab

- 1 TSP 的整数模型 = 指派问题（出入度约束）+ “连成一个圈”。只保留出入度约束，解是一堆互不相连的小回路，下界很松（本章例子里低了 22%）。精确解可以用 Held-Karp 动态规划， $O(n^2 2^n)$ ，只适合二十个点以内。
- 2 子回路消除约束有指数多个。做法是割平面：解 LP，分离被违反的约束，加入，再解。分离用最大流：把 LP 的弧流量当容量，仓库到某点的最大流小于 1，汇侧的最小割点集就是被违反的约束。分数解也适用。

- 3 本章例子里割平面循环 4 轮、6 条约束就把下界从 198 推到 255，等于最优；完整的约束有 247 条。求解器里对应的是惰性约束。
- 4 MTZ 用站序号变量把约束压成多项式个，规模小、好写，但其中的 Big-M 系数让 LP 很弱，只比指派松弛好一点。下界顺序：指派松弛 $\leq$ MTZ $\leq$ 子回路约束 $\leq$ 最优。
- 5 CVRP 是前几章的拼装：出入度（指派）、载重（装载与背包）、子回路。容量割 $\lceil d(S)/Q \rceil$ 把装箱下界和子回路约束合在一条不等式里。分组的好坏决定路线：扫描法比最优多 16%。
- 6 构造启发式加局部搜索快，但没有质量保证。2-opt 停在局部最优，本章例子里比最优长 2.7%。用下界可以判断离最优最多还差多少。

术语表

中文	English	在本章的含义
旅行商问题	traveling salesman problem (TSP)	从仓库出发走遍所有点再回来，总距离最小
子回路	subtour	不经过仓库、只连接一部分点的闭合小圈
子回路消除约束	subtour elimination constraint (SEC)	$\sum_{S \text{ 内的弧}} x \leq S - 1$
分离	separation	在当前 LP 解里找一条被违反的约束；这里用最大流
惰性约束	lazy constraints	不预先写入模型，解出后检查、需要时再加
MTZ 约束	Miller–Tucker–Zemlin	用站序号变量 u 排除子回路的紧凑模型
Held–Karp	Held–Karp	旅行商的动态规划；也指子回路 LP 给出的下界
车辆路径问题	vehicle routing problem (VRP, CVRP)	多辆车从仓库出发服务客户，带载重的叫 CVRP
容量割	rounded capacity inequality	进入客户集合 S 的弧数 $\geq \lceil d(S)/Q \rceil$
扫描法	sweep algorithm	按相对仓库的角度排序、依次装车的分组启发式
最近邻、2-opt	nearest neighbor, 2-opt	构造启发式；反转一段的局部搜索
局部最优	local optimum	邻域里没有更好解的解

对照教材

- Toth & Vigo (编) , *Vehicle Routing: Problems, Methods, and Applications* (第 2 版 , SIAM 2014) : CVRP 的模型、容量割、分支切割与启发式。Applegate、Bixby、Chvátal、Cook , *The Traveling Salesman Problem: A Computational Study* (Princeton , 2006) : 割平面与分离的实战。
- Wolsey , *Integer Programming* (第 2 版 , Wiley 2020) : TSP 的模型与子回路约束、惰性约束的思路 ; Cook、Cunningham、Pulleyblank、Schrijver , *Combinatorial Optimization* : 旅行商与最小割分离。
- Dantzig、Fulkerson、Johnson , “Solution of a large-scale traveling-salesman problem” (1954) : 子回路约束与割平面的起点。Miller、Tucker、Zemlin , “Integer

programming formulation of traveling salesman problems” (J. ACM , 1960) 。
Desrochers & Laporte (1991) : MTZ 的提升。

- Held & Karp , “A dynamic programming approach to sequencing problems” (1962) ; Held & Karp (1970) : 1-tree 下界。Croes (1958) : 2-opt。Christofides (1976) : 1.5 倍的近似算法。Gillett & Miller , “A heuristic algorithm for the vehicle-dispatch problem” (1974) : 扫描法。

本章 Lab

代码包 `ch08-tsp-vrp-lab.zip` 随本章一起发放。依赖是 `numpy`、`scipy`、`pytest` ; 判分用暴力枚举、完整子回路约束的 LP 和 `linprog` , 不依赖你的代码。

练习	内容	对应
ex1	旅行商 : Held-Karp 动态规划、指派松弛与拆回路 ; 先填 4 个预测	8.1
ex2	子回路消除 : 用最大流分离, 手写根节点的割平面循环 ; 停下时的值必须等于完整约束的 LP	8.2
ex3	MTZ : 建模并求 LP , 验证 指派 $\leq$ MTZ $\leq$ 子回路 $\leq$ 最优	8.3
ex4	CVRP : 检查路线、子集动态规划求最优、扫描法	8.4
ex5	审 5 个 AI 写的旅行商函数 : 不看源码, 只靠运行结果归类问题	8.1
挑战	2-opt 局部搜索 : 按规定的扫描顺序, 步数与页面例子一致	8.5

```
pip install -r requirements.txt
pytest -q # 全部练习
pytest tests/test_ex2.py -q # 单个练习

# 确认判分本身没问题 : 参考答案应当全部通过
LAB_TARGET=solutions pytest -q
```

练习 1 要求先在 `PREDICTION` 里写下预测再运行 ; 跑完测试后, 终端最后会把你的预测和实际结果并排列出来。

下一章

第二部分到这里结束：指派、网络、容量、路线，都是“把东西分给谁、怎么走”。第三部分转向“什么时候做”：提升机、站点、堆垛设备都是要排队的机器。下一章先学调度的语言：机器环境、约束、目标三段式记号，Cmax、加权完工时间、延迟等目标，以及一张复杂度地图，告诉你哪些调度问题容易、哪些很难。

8.8 自测

8 道题，覆盖本章的全部要点。每题选完立即给出解析。

Q1 只保留出入度约束的旅行商松弛，最优解为什么常常是一堆小回路？

这个松弛就是指派问题：每个点选一个后继，只保证有一进一出，不要求连成一个圈，所以最优解偏向短小的圈

因为距离矩阵不对称

因为点太多

因为求解器不够好

正确。 指派问题的解是一个置换，置换分解成若干个圈。子回路消除约束就是用来规定“只有一个圈”的。

Q2 子回路消除约束有 $2^{(n-1)} - n$ 个，为什么可以放心用？

因为求解器会一次性加载它们

不用一次写完：先解 LP，分离出被违反的约束，加入，再解，实际只需要很少几条

因为大多数约束是多余的，可以直接删掉

因为 n 不会很大

正确。 关键是分离能高效地做（一次最大流），这样指数多的约束就可以按需添加。本章例子里 247 条只用了 6 条。

Q3 LP 解 x 里，仓库到点 t 的最大流 (x 当容量) 小于 1，说明什么？

t 是最优回路上的最后一个点

点 t 到不了仓库

有一组包含 t 的点被孤立在外面，进入它的弧流量之和不足 1，对应一条被违反的子回路约束

LP 无解

正确。 出入度约束成立时， S 内部的弧数 = $|S|$ - 进入 S 的弧数。子回路约束等价于进入 S 的弧流量至少为 1，也就是仓库到 S 中每个点都能送过 1 个单位。最小割的汇侧就是被违反的 S 。

Q4 割平面循环停下时（再也分离不出被违反的约束），LP 的值是？

0

指派松弛的值

一定等于最优值

加入全部子回路约束的 LP 值，与加入的顺序无关；它是下界，可能低于最优，也可能带分数解

正确。 停下说明所有子回路约束都满足，LP 值就是完整约束下的最优。本章例子里它恰好等于最优 255；一般实例上它是一个很强的下界，差距交给分支。

Q5 MTZ 模型的约束数只有 m^2 级，LP 下界为什么弱？

约束里有 Big-M 系数 m ： x 取小分数时约束几乎不起作用

因为它不排除子回路

因为 u 必须是整数

因为变量太多

正确。 $x_{ij} = 0$ 时约束必须永远成立，所以系数取到 m ；LP 里 x 很小时约束几乎松掉。这是第 3 章讲的 Big-M 的典型代价。

Q6 下面哪个顺序对任何实例都成立？

最优 $\leq$ 子回路约束的 LP $\leq$ MTZ $\leq$ 指派松弛

指派松弛 $\leq$ MTZ 的 LP $\leq$ 子回路约束的 LP $\leq$ 最优

MTZ $\leq$ 指派松弛 $\leq$ 最优 $\leq$ 子回路约束

四者相等

正确。 松弛越多、约束越弱，下界越低；子回路约束是最强的，但仍是松弛，所以不超过最优。页面和 Lab 在随机实例上验证了这个顺序。

Q7 CVRP 的容量割“进入 S 的弧数 $\geq \lceil d(S)/Q \rceil$ ”，为什么说它包含了子回路约束？

因为它更弱

因为它写在同一个表里

$d(S) > 0$ 时 $\lceil d(S)/Q \rceil \geq 1$ ，这就是“必须有弧从外面进入 S”，也就是子回路约束；而且它还带了装箱的下界

因为 Q 总是 1

正确。 $\lceil d(S)/Q \rceil$ 是装 S 里的托盘至少要的车数，也就是第 7 章装箱的下界。它至少为 1，所以不经过仓库的小圈同样被排除。

Q8 2-opt 停下来时，得到的回路是？

最近邻回路

不合法的回路

最优回路

对“反转一段”这类邻域的局部最优：任何一次反转都不能再缩短，但可能比全局最优长

正确。 本章例子里 2-opt 停在 262，比最优 255 长 2.7%。要跳出局部最优，需要更大的邻域或者允许暂时变差的方法（第六部分）。

8 / 8 本章掌握得不错，可以去做 Lab 了。

数据说明：本章的全部数字由页面内置的 Held-Karp、单纯形法、最大流分离与割平面循环、MTZ 的 LP、子集动态规划、最近邻与 2-opt 实时计算，并与暴力枚举、完整子回路约束的 LP 和 SciPy 的 `linprog` 逐项对拍（环境：SciPy 1.17.1 · HiGHS 1.12.0）：随机实例上的最优回路、三种模型的下界、割平面循环的终点、CVRP 的最优值，都与 Lab 的判分数据同源。